

FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering Academic Year 2025-26

Rubrics for Lab Experiments

**Class : B.E. Computer
Engineering**

Subject Name :DC Lab

Semester : VIII

Subject Code : CSL801

Practical No:	5
Title:	To implement Election Algorithms .
Date of Performance:	14/11/25
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation:

Performance Indicator	Below average	Average	Good	Excellent	Marks
On time Submission (2)	Not submitted (0)	Submitted after deadline (1)	Early or on time submission(2)	---	
Test cases and output (4)	Incorrect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

Signature of the Teacher:

LAB 5

Aim: To implement Election Algorithms

Theory:

In distributed systems, election algorithms are used to elect a leader or coordinator node among a group of nodes. The leader node is responsible for coordinating the activities of other nodes, managing resources, and making decisions on behalf of the group.

There are various election algorithms used in distributed systems, and the choice of algorithm depends on the requirements of the system, such as fault-tolerance, scalability, and availability.

One commonly used election algorithm is the Bully algorithm, which is a centralized algorithm. In this algorithm, the node with the highest priority becomes the leader, and if the leader fails, the next highest priority node takes over.

Another algorithm is the Ring algorithm, which is a decentralized algorithm. In this algorithm, nodes are arranged in a ring, and each node sends a message to its neighbor indicating its willingness to become the leader. The node with the highest priority becomes the leader.

A third algorithm is the Paxos algorithm, which is fault-tolerant and can handle network failures. In this algorithm, nodes propose a value to be chosen as the leader, and if the majority of nodes accept the proposal, the value is chosen as the leader.

These election algorithms ensure that a leader is elected in a fair and efficient manner, and they provide fault-tolerance and availability in distributed systems.

Bully Algorithm

The bully Algorithm proposed by Garcia-Molina follows the following algorithm

- When a process notices that the coordinator is no longer responding to requests, it initiates an election.
- A process, P, holds an election as follows:
 1. P sends an ELECTION message to all processes with higher numbers
 2. If no one responds, P wins the election and becomes the coordinator
 3. If one of the higher-up's answers, it takes over. P's job is done
- If a process can get an ELECTION message from one of its lower-numbered colleagues.

The message arrives => the receiver sends an OK message back to the sender to indicate that he is alive and will take over.

The receiver then holds an election, unless it is already holding one.

- Eventually, all processes give up but one does not give up and that one is the new coordinator.

It announces its victory by sending all processes a message telling them that starting immediately it is the new coordinator.

- If a process that previously went down came back up, it holds an election. If it happens to be the highest-numbered process currently running, it will win the election and take over the coordinator's job.
- Thus, the biggest guy in town always wins, hence the name "bully algorithm."

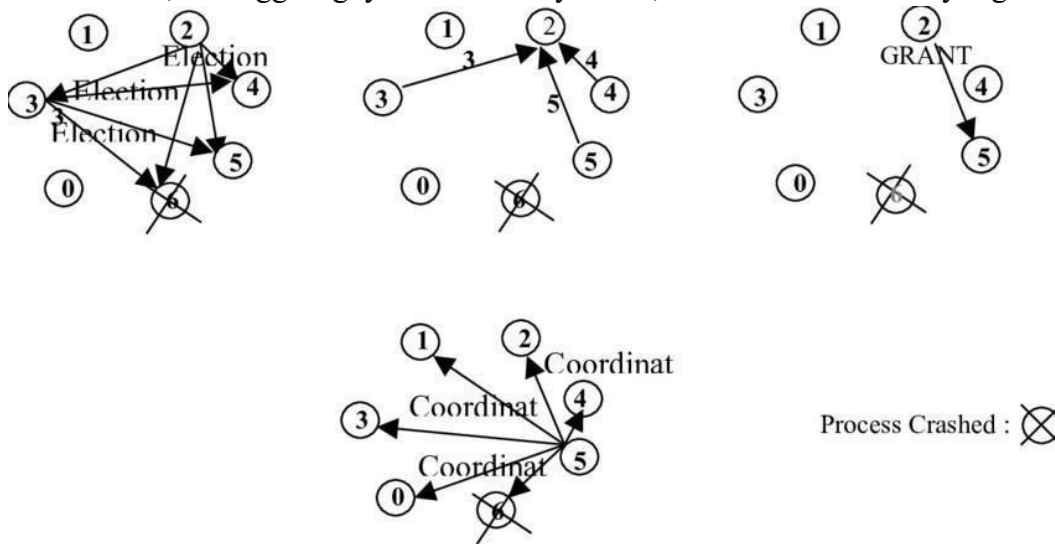


Fig. Bully Algorithm

Code & Output:

BullyElection.java

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicBoolean;

public class BullyElection {
    public static void main(String[] args) throws Exception {
        final int N = 6; // number of nodes (IDs 1..N)
        final List<BlockingQueue<Message>> queues = new ArrayList<>();

        for (int i = 0; i < N + 1; i++)
            queues.add(new LinkedBlockingQueue<>()); // index by id

        List<Node> nodes = new ArrayList<>();

        for (int id = 1; id <= N; id++) {
            Node node = new Node(id, N, queues);
            nodes.add(node);
            new Thread(node, "Node-" + id).start();
        }

        Thread.sleep(2000);

        System.out.println("\n--- Simulate coordinator failure ---\n");

        // fail the highest ID
        int failId = N;
        for (Node node : nodes) {
            if (node.id == failId) {
```

```

        node.fail();
        break;
    }
}

Thread.sleep(8000);

System.out.println("\n--- Recovering failed node ---\n");

for (Node node : nodes) {
    if (node.id == failId) {
        node.recover();
        break;
    }
}

Thread.sleep(8000);

for (Node node : nodes) node.shutdown();

System.out.println("\nSimulation finished.");
}
}

enum MsgType { ELECTION, OK, COORDINATOR, HEARTBEAT }

class Message {
    final MsgType type;
    final int senderId;
    final int coordinatorId;

    Message(MsgType type, int senderId) {
        this(type, senderId, -1);
    }

    Message(MsgType type, int senderId, int coordinatorId) {
        this.type = type;
        this.senderId = senderId;
        this.coordinatorId = coordinatorId;
    }

    public String toString() {
        if (type == MsgType.COORDINATOR) {
            return type + "(from " + senderId + ", coord=" + coordinatorId + ")";
        } else {
            return type + "(from " + senderId + ")";
        }
    }
}

class Node implements Runnable {
    final int id;
    final int N;
    final List<BlockingQueue<Message>> queues;
    final BlockingQueue<Message> myQueue;
    volatile int currentCoordinator = -1;

    final AtomicBoolean alive = new AtomicBoolean(true);
    final AtomicBoolean running = new AtomicBoolean(true);

    final Random rnd = new Random();
    volatile long lastHeartbeat = System.currentTimeMillis();

    Node(int id, int N, List<BlockingQueue<Message>> queues) {
        this.id = id;
        this.N = N;
    }
}

```

```

this.queues = queues;
this.myQueue = queues.get(id);

if (id == N) currentCoordinator = N;
}

void send(int toId, Message m) {
    try {
        queues.get(toId).put(m);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

void broadcast(Message m) {
    for (int i = 1; i <= N; i++) {
        if (i != id) send(i, m);
    }
}

void startElection() {
    if (!alive.get()) return;

    System.out.println(logPrefix() + "Starting election");

    boolean higherExists = false;

    for (int j = id + 1; j <= N; j++) {
        higherExists = true;
        send(j, new Message(MsgType.ELECTION, id));
    }

    if (!higherExists) {
        becomeCoordinator();
        return;
    }

    long waitUntil = System.currentTimeMillis() + 1000;
    boolean gotOk = false;

    while (System.currentTimeMillis() < waitUntil) {
        Message m = myQueue.poll();

        if (m == null) {
            try {
                Thread.sleep(50);
            } catch (InterruptedException ignored) {}
            continue;
        }

        if (m.type == MsgType.OK) {
            gotOk = true;
            break;
        }
    }

    if (!gotOk) {
        becomeCoordinator();
    }
}

void becomeCoordinator() {
    currentCoordinator = id;
    System.out.println(logPrefix() + "BECAME COORDINATOR");
    broadcast(new Message(MsgType.COORDINATOR, id, id));
}

```

```

void fail() {
    System.out.println(logPrefix() + " FAILING (crashed)");
    alive.set(false);
}

void recover() {
    System.out.println(logPrefix() + " RECOVERED");
    alive.set(true);
    startElection();
}

void shutdown() {
    running.set(false);
}

String logPrefix() {
    return "[Node " + id + "] ";
}

@Override
public void run() {
    while (running.get()) {
        if (!alive.get()) {
            try { Thread.sleep(100); } catch (InterruptedException ignored) {}
            continue;
        }

        Message m = myQueue.poll();
        if (m != null) {
            switch (m.type) {
                case ELECTION:
                    if (alive.get()) {
                        send(m.senderId, new Message(MsgType.OK, id));
                        startElection();
                    }
                    break;

                case COORDINATOR:
                    currentCoordinator = m.coordinatorId;
                    System.out.println(logPrefix() + "New coordinator is " + currentCoordinator);
                    break;

                default:
                    break;
            }
        }

        // heartbeat to coordinator
        if (id != currentCoordinator && currentCoordinator != -1) {
            long now = System.currentTimeMillis();
            if (now - lastHeartbeat > 2000) {
                System.out.println(logPrefix() + "Coordinator missing. Starting election.");
                startElection();
            }
        }

        try { Thread.sleep(50); } catch (InterruptedException ignored) {}
    }
}

```

```
Hallowmest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp5
> java BullyElection
```

```
--- Simulate coordinator failure ---
```

```
[Node 6] FAILING (crashed)
```

```
--- Recovering failed node ---
```

```
[Node 6] RECOVERED
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 3] New coordinator is 6
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 2] New coordinator is 6
[Node 4] New coordinator is 6
[Node 4] Coordinator missing. Starting election.
[Node 1] New coordinator is 6
[Node 5] New coordinator is 6
[Node 5] Coordinator missing. Starting election.
[Node 5] Starting election
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 4] Starting election
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 3] New coordinator is 6
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] New coordinator is 6
[Node 5] Coordinator missing. Starting election.
[Node 5] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] New coordinator is 6
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 4] New coordinator is 6
```

```
[Node 4] Coordinator missing. Starting election.
[Node 4] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 2] New coordinator is 6
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 3] New coordinator is 6
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] New coordinator is 6
[Node 5] Coordinator missing. Starting election.
[Node 5] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] New coordinator is 6
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 4] New coordinator is 6
[Node 4] Coordinator missing. Starting election.
[Node 4] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 2] New coordinator is 6
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 6] Starting election
[Node 3] New coordinator is 6
[Node 6] BECAME COORDINATOR
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] New coordinator is 6
[Node 6] Starting election
[Node 5] Coordinator missing. Starting election.
[Node 6] BECAME COORDINATOR
[Node 5] Starting election
[Node 1] New coordinator is 6
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] Starting election
```

```
[Node 6] BECAME COORDINATOR
[Node 4] New coordinator is 6
[Node 4] Coordinator missing. Starting election.
[Node 4] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 2] New coordinator is 6
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 3] New coordinator is 6
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] New coordinator is 6
[Node 5] Coordinator missing. Starting election.
[Node 5] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] New coordinator is 6
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 4] New coordinator is 6
[Node 4] Coordinator missing. Starting election.
[Node 4] Starting election
[Node 2] New coordinator is 6
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 3] New coordinator is 6
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] New coordinator is 6
[Node 5] Coordinator missing. Starting election.
[Node 5] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] New coordinator is 6
[Node 6] Starting election
```



```

[Node 1] New coordinator is 6
[Node 6] Starting election
[Node 1] Coordinator missing. Starting election.
[Node 6] BECAME COORDINATOR
[Node 1] Starting election
[Node 2] New coordinator is 6
[Node 4] New coordinator is 6
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 4] Coordinator missing. Starting election.
[Node 4] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 3] New coordinator is 6
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] New coordinator is 6
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] Coordinator missing. Starting election.
[Node 5] Starting election
[Node 1] New coordinator is 6
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 2] New coordinator is 6
[Node 4] New coordinator is 6
[Node 4] Coordinator missing. Starting election.
[Node 4] Starting election
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 3] New coordinator is 6
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] New coordinator is 6
[Node 5] Coordinator missing. Starting election.
[Node 5] Starting election

```

```

[Node 1] New coordinator is 6
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 4] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 2] New coordinator is 6
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] New coordinator is 6
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 3] New coordinator is 6
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] New coordinator is 6
[Node 5] Coordinator missing. Starting election.
[Node 5] Starting election
[Node 1] New coordinator is 6
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 4] Coordinator missing. Starting election.
[Node 4] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 2] New coordinator is 6
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 1] New coordinator is 6
[Node 6] Starting election
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] BECAME COORDINATOR
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 3] Starting election

```

```
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 4] New coordinator is 6
[Node 4] Coordinator missing. Starting election.
[Node 4] Starting election
[Node 1] New coordinator is 6
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 5] New coordinator is 6
[Node 5] Coordinator missing. Starting election.
[Node 5] Starting election
[Node 1] New coordinator is 6
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 2] Coordinator missing. Starting election.
[Node 2] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 3] New coordinator is 6
[Node 3] Coordinator missing. Starting election.
[Node 3] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] New coordinator is 6
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR
[Node 1] New coordinator is 6
[Node 1] Coordinator missing. Starting election.
[Node 1] Starting election
[Node 6] Starting election
[Node 6] BECAME COORDINATOR

Simulation finished.
[Node 5] BECAME COORDINATOR
[Node 2] BECAME COORDINATOR
[Node 3] BECAME COORDINATOR
```

Conclusion :

The Bully Election Algorithm ensures reliable coordinator selection in distributed systems by allowing the highest-priority active process to take control. When a failure occurs, remaining processes initiate an election to determine the new leader. This approach provides simplicity, fault tolerance, and efficient leader recovery, ensuring smooth system coordination.

Postlab Questions:.

1. Role of a Co-ordinator in distributed systems

A co-ordinator in distributed systems manages & synchronizes the activities of multiple processes. It handles tasks such as resource allocation, communication control, task scheduling and resolving conflicts. The co-ordinator helps in maintaining consistency, reliability and state without deadlocks. Overall it acts as a central authority that streamlines operations & ensures smooth & efficient functioning.

2. Compare Bully and Ring

The bully algorithm selects the highest-ID active process coordinator, allowing any process to start an election. It is faster, but generates more message overhead based on process priorities. The Ring algorithm arranges processes in a logical ring, passing election messages sequentially. It reduces message overhead and avoids priority-based dominance but can be slower. Bully is centralized & aggressive while Ring is decentralized and order